

CS & IT ENGINEERING

Programming in C

Functions and Storage Classes
Lec- 02



By- Pankaj Sharma sir



TOPICS TO BE
COVERED



Storage Classes

Storage class

block → {
 }
 }

(i) scope : part of code in which a variable is visible (can be accessed directly).

(ii) Lifetime :

(iii) Default value : If the variable is not initialized then what is its default value.

for ex :- void main() {

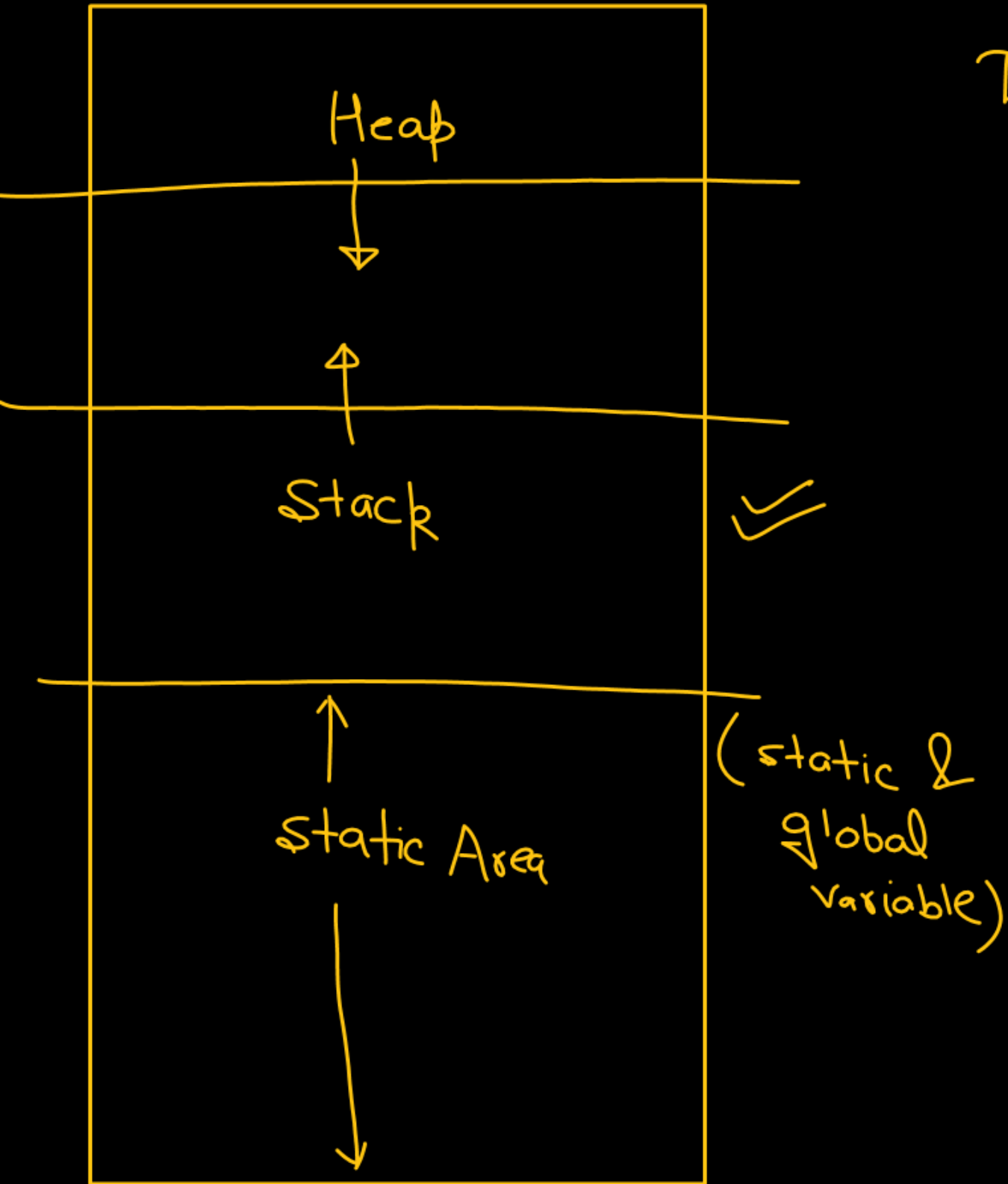
int a;

printf("%d", a);

}

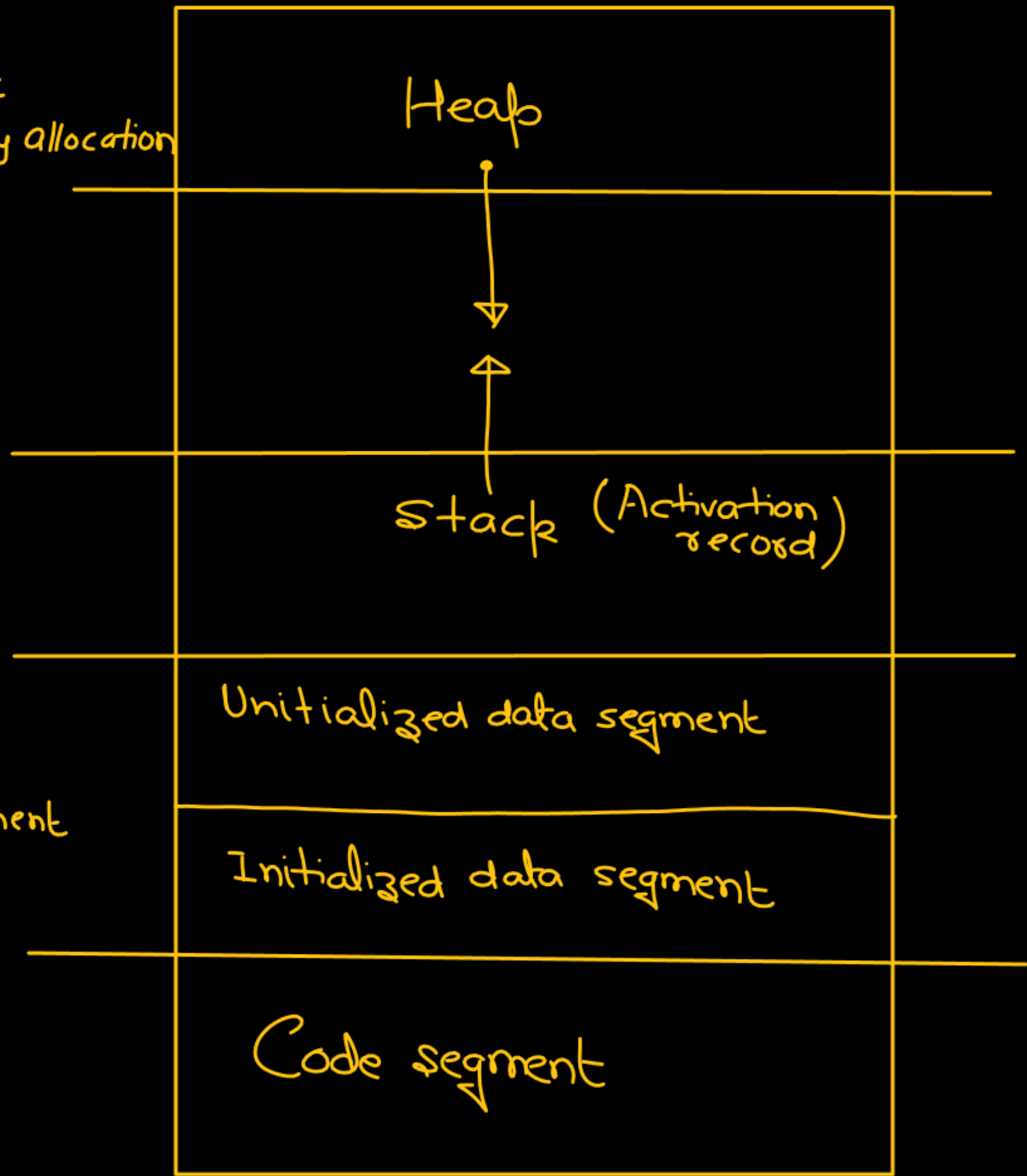
Garbage

(iv) Storage area : Where the variable is stored



Dynamic
Memory allocation

data
segment



Storage class

(i) auto : by default a variable declared inside a function is auto variable (local variable)

```
void main(){
```

```
    int a = 10, b = 20, result;
```

```
    result = Add(a, b);
```

```
    printf("%d", result);
```

```
}
```

a, b,
result
are
local to
main

```
int Add(int x, int y)
```

```
{
```

```
    int sum;
```

```
    sum = x + y;
```

```
    return sum;
```

```
}
```

local
to
Add
function

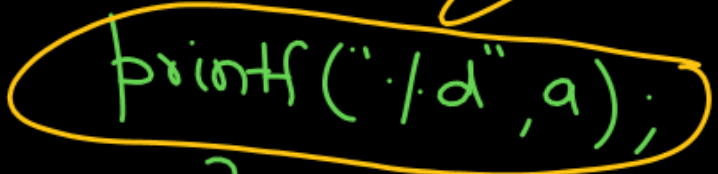
(i) scope : block in which they are declared.

```
void main() {  
    int a;  
    printf("%d", a);  
}
```



```
void fun() {  
    printf("%d", a);  
}
```

error



```
void fun ( )  
{  
    int a = 10;  
    printf("%d", a);  
}
```

```
void main(){  
    int a = 1;  
    printf("%d", a);  
}
```

```
void fun(){  
    auto int a = 10;
```

Why auto: They are created automatically once you enter the block in which they are declared & they are destructed automatically once you exit the block in which they are declared.

1 1 1

void fun(){

→ 1 int i = 0;

2 ++i;

3 printf("%d", i);

4 }

void main(){

1 fun();

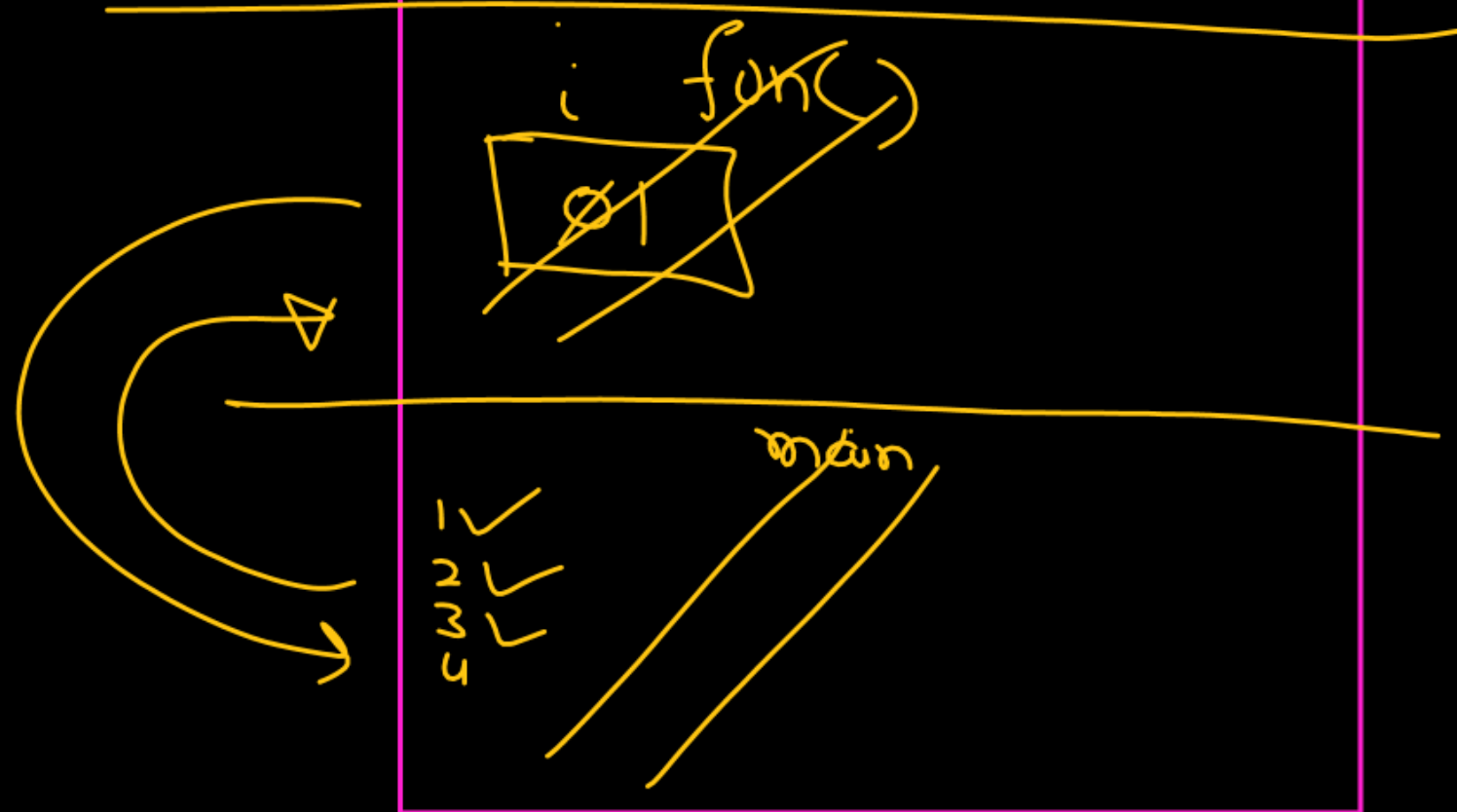
2 fun();

3 fun();

4 }

~~int a = 10~~

00001010



auto

- (1) scope : block
- (2) Lifetime: block
- (3) Default value: Garbage Value
- (4) Storage Area: stack

block-scope {
scope
}

```
void main() {  
    int a;  
    printf("%d", a);  
}
```

```
void main(){
```

```
  int a = 10;
```

```
  ++a;
```

```
  printf("%d", a); → 11
```

```
{
```

```
  int b, a = 3;
```

```
  printf("%d", a);
```

```
  ++a;
```

```
}
```

```
  ++a;
```

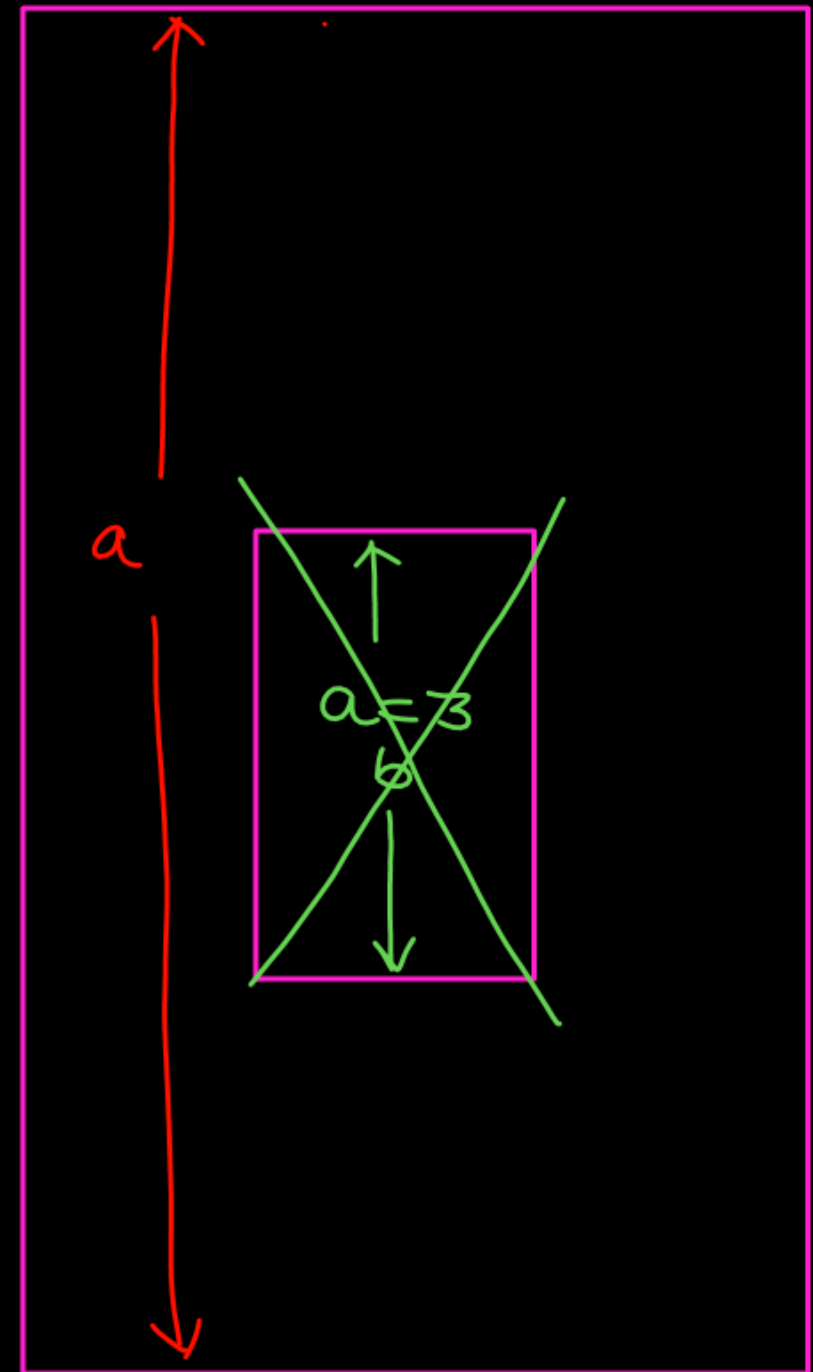
```
  printf("%d", a); 12
```

```
}
```



3

11 3 12

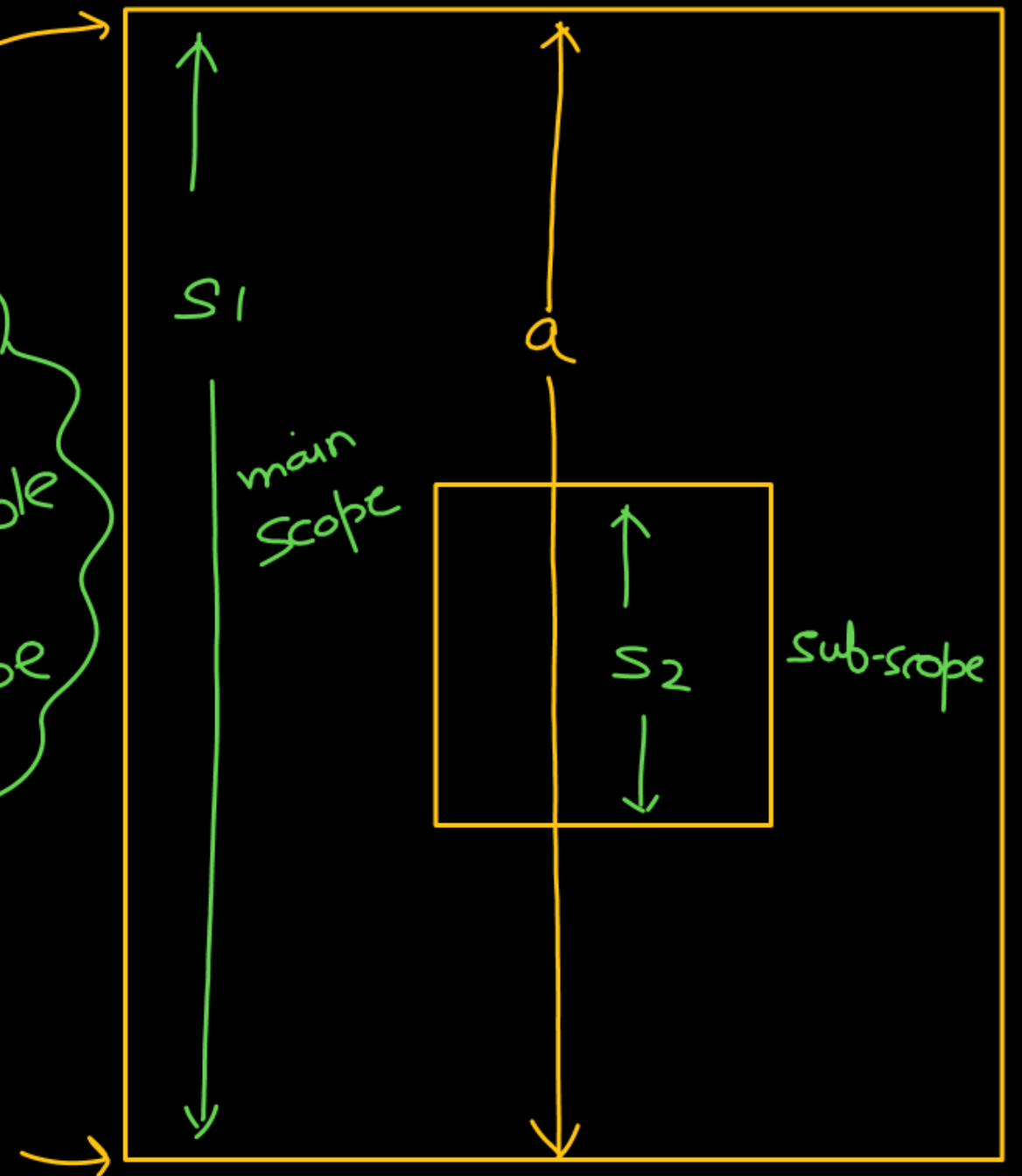


11 12 13

```
void main(){  
    int a = 10;  
    ++a; ✓  
    printf("/d", a);  
    {  
        ++a; ✓  
        printf("/d", a);  
    }  
    ++a; ✓  
    printf("/d", a);  
}
```



main scope
variable
are accessible
in
sub-scope



```

void main() {
    int a = 0;
    ++a;
    printf("%d", a);

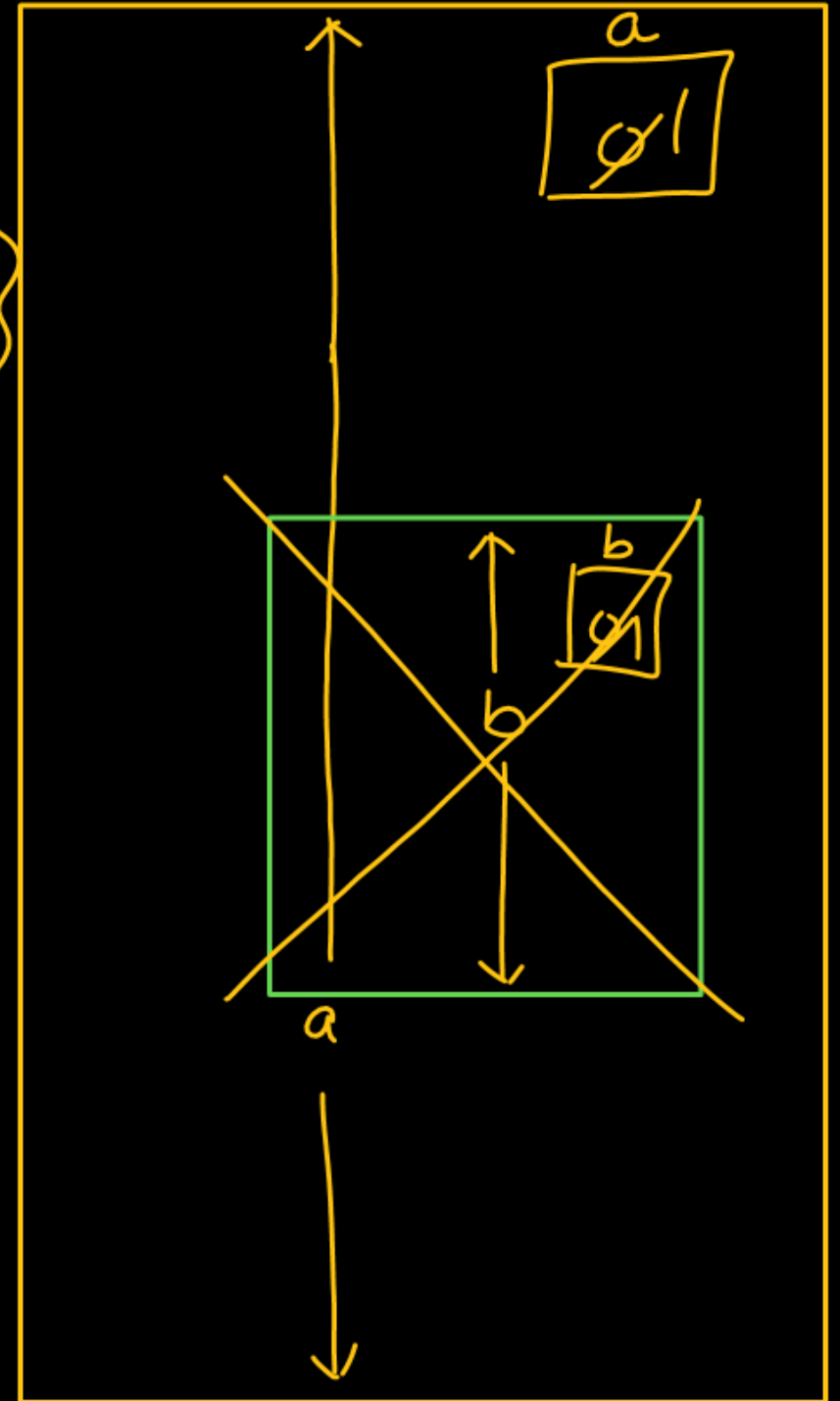
    {
        int b = 0;
        ++b;
        printf("%d", b);
    }

    ++b;
    ++a;
    printf("%d %d", a, b);
}

```

Sub scope
var.
are not accessible
in
main-scope

Error



register

* Same as auto except storage area.

void main(){
Command auto int i;
=

void main(){
register int i; → request

register → stack/CPU register

- block scope
- Garbage
- block lifetime

```
void main(){
```

~~register~~ int i → request is granted

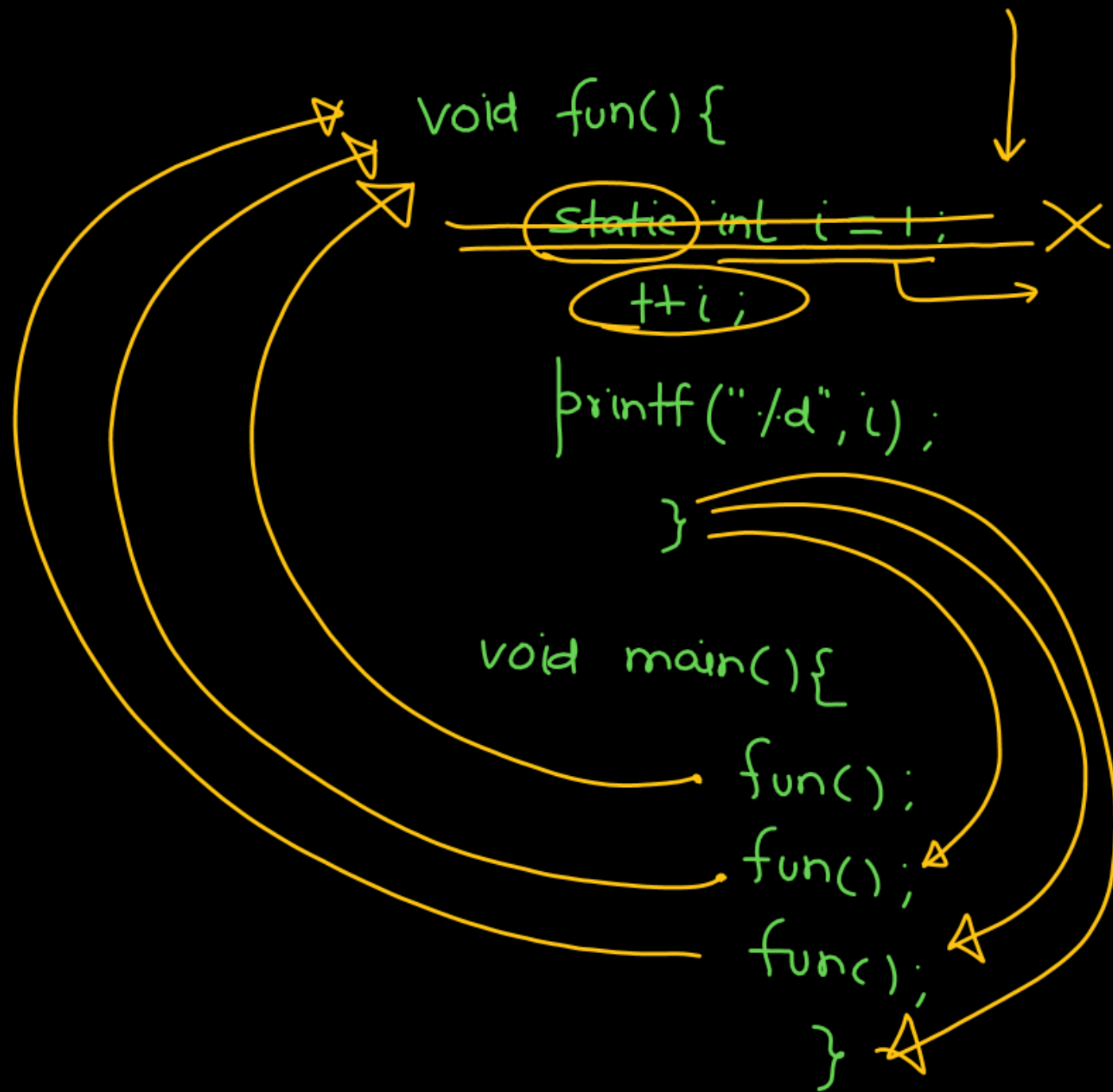
→ requested is rejected → auto

CPU register



Static variables

- * They are created once for a program and remain in memory till end of the program.
- * Lifetime : static variable is throughout the program
- * Scope : block in which they are declared
- * Default value : 0
- * storage Area : data segment (static Area)



No re-initialization of static var.

i [1 2 3 4]

[2 3 4]

```
void fun(){
```

```
    static int i;
```

```
    ++i;
```

```
    printf("%d", i);
```

```
}
```

scope:

within
block

```
void main(){
```

```
    int a = 10;
```

```
    printf("%d", a + i);
```

```
}
```

Error

Global variable

global
int i;

void f(){

}

void g(){

}

void main(){

}

default
value $\Rightarrow 0$

int i; $\rightarrow$ global variable

```
void f(){
```

```
    ++i; ✓
```

```
}
```

```
void g(){
```

```
    ++i; ✓
```

```
}
```

```
void main(){
```

```
    ++i; ✓
```

```
    printf("%d", i);  
}
```



```
void f() {  
    ++i;  
    printf("/d", i);  
}  
int i = 0;  
void g() {  
    ++i;  
    printf("/d", i);  
}  
void main() {  
    ++i;  
    f();  
    g();  
    printf("/d", i);  
}
```

CE ?

To avoid compilation
Error

↳ forward declaration

```
void f() {  
    ++i;  
    printf("/d", i);  
}
```

```
int i = 0;
```

```
void g() {  
    ++i;  
    printf("/d", i);  
}
```

```
void main() {  
    ++i;  
    f();  
    g();  
    printf("/d", i);  
}
```

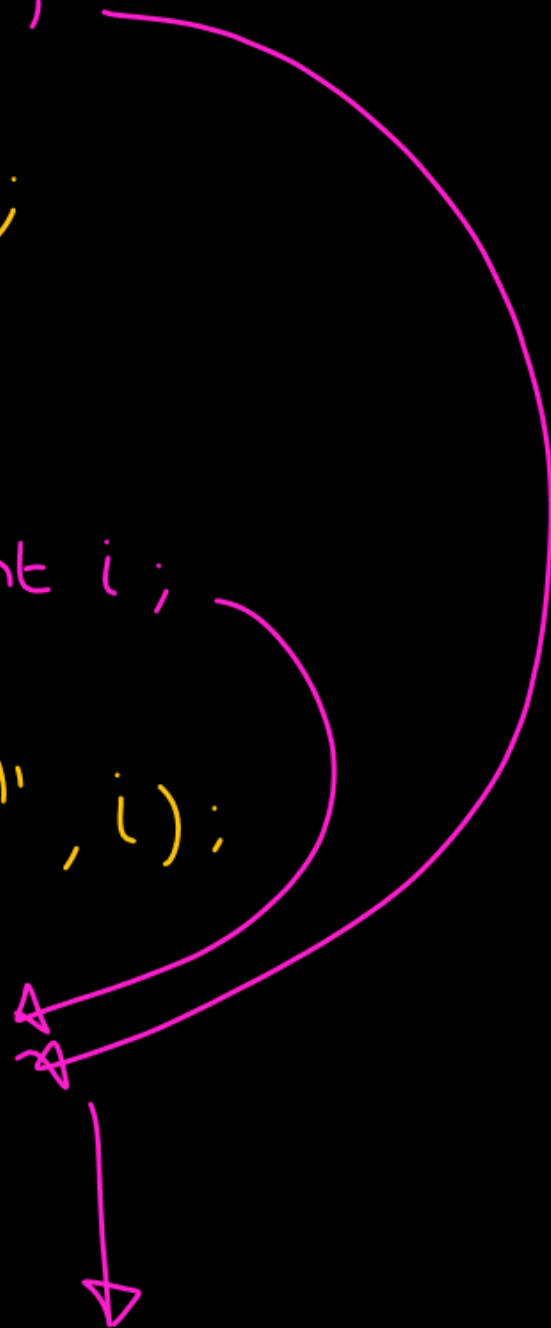
No memory is created
forward dec. of global var. i

```
void f() {  
    extern int i;  
    ++i;  
    printf("/d", i);  
}
```

```
void g() {  
    extern int i;  
    ++i;  
    printf("/d", i);  
}
```

```
int i = 0;
```

```
void main() {  
    ++i;  
    f();  
    g();  
    printf("/d", i);  
}
```



```
void f(){
    extern int i;
    ++i;
    ==
}
```

```
void g(){
    extern int i;
    ++i;
    ==
}
```

```
void main(){
    extern int i;
    ++i;
    f();
    g();
    ==
}
```

```
int i = 0;
```

forward
dec.
No memory
is
created

⇒

```
extern int i;
void f(){
    ++i;
    ==
}
void g(){
    ++i;
    ==
}
void main(){
    ++i;
    f();
    g();
}
int i = 0;
```

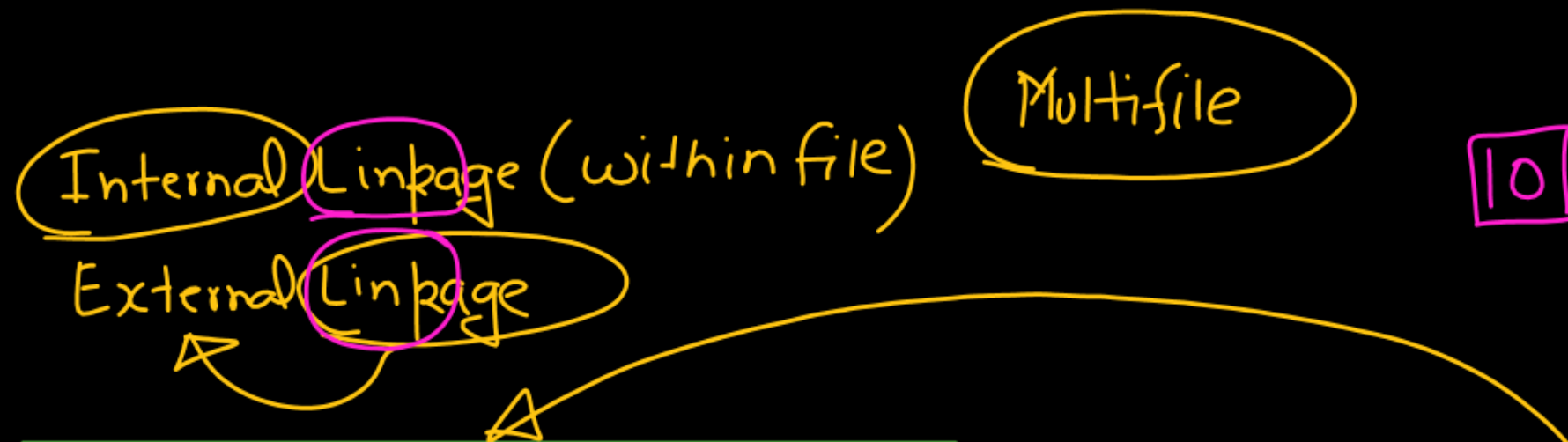
define

int i = 0;

Global $\begin{cases} \rightarrow \text{dec.} \\ \rightarrow \text{def.} \end{cases} \text{diff}$

defination: With def. memory is created.

but with dec. no memory is created.



```
int i = 10;  
void main()  
{  
  
}
```

1.c

```
extern int i;  
void f()  
{  
  
}  
void g()  
{  
  
}
```

2.c

